LECTURE

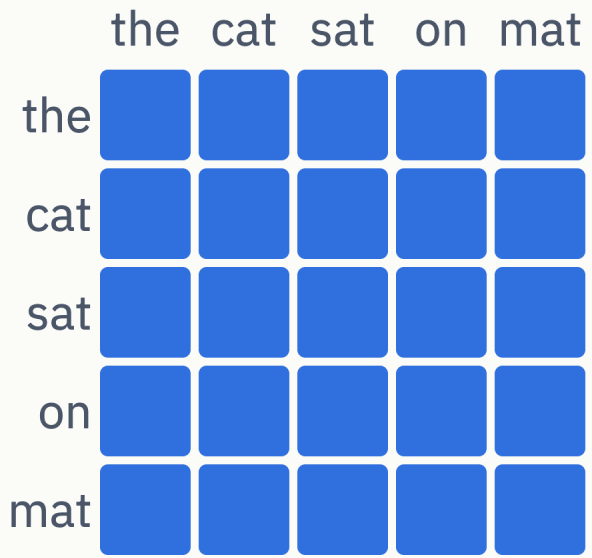
From BERT to Jev-like models

Encoders, decoders, and how a model can choose from any list of options

BERT · GPT-2 · ModernBERT · Von

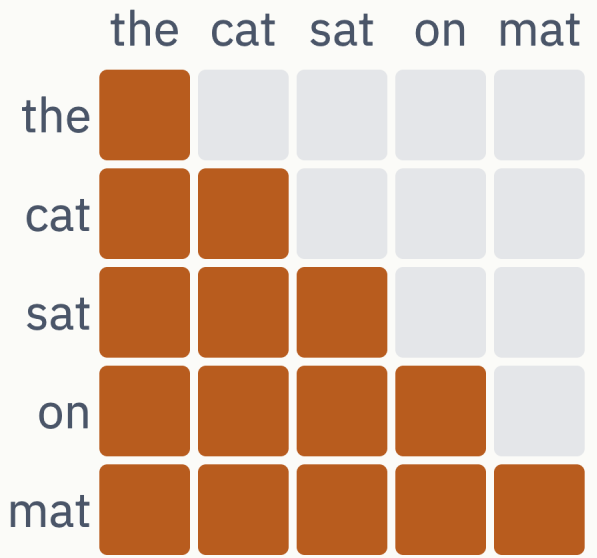
Who can read whom

Encoder (BERT)



Reads **every** token.

Decoder (GPT)



Reads only **earlier** tokens.

Row = the token that reads. Grey = blocked.

Same size, different mask

	BERT-base	GPT-2 small
Layers	12	12
Hidden size	768	768
Attention heads	12	12
Parameters	110M	124M
Attention	Both directions	Left only
Vocabulary	30,522	50,257

BERT input: three vectors per token

Token	[CLS]	my	dog	is	cute	[SEP]	he	likes	play	##ing	[SEP]
Segment	A	A	A	A	A	A	B	B	B	B	B
Position	0	1	2	3	4	5	6	7	8	9	10

```
x_i = token_emb[tok_i] + segment_emb[A or B] + position_emb[i]
```

[CLS] = summary

[SEP] = separator

##ing = sub-word piece

BERT task 1: fill in the masked word

my dog is hairy → BERT selects 15% of tokens. Here: **hairy**.

How often	Input to BERT	Target
80%	my dog is [MASK]	hairy
10%	my dog is apple	hairy
10%	my dog is hairy	hairy

Loss: **only the selected positions**. ModernBERT masks **30%**.

BERT task 2: does B follow A?

```
[CLS] the man went to [MASK] store [SEP] he bought a gallon [MASK]
milk [SEP]
```

IsNext

```
[CLS] the man [MASK] to the store [SEP] penguin [MASK] are flight ##less
birds [SEP]
```

NotNext

A 2-class head on **[CLS]** predicts the label. ModernBERT drops this task.

GPT task: predict the next token

Sentence: **The cat sat on the mat**

Position	1	2	3	4	5
Reads	The	…cat	…sat	…on	…the
Predicts	cat	sat	on	the	mat

Loss at **every position**. The causal mask hides each answer, so **one pass trains all 5**.

One training sample each

BERT (masked LM)

input	[CLS]	my	dog	is	[MASK]	[SEP]
label	-100	-100	-100	-100	hairy	-100

-100 = ignored by the loss.

GPT (next token)

input	The	cat	sat	on	the
label	cat	sat	on	the	mat

Labels = inputs shifted left by one.

Pre-training data

Model	Data	Size
BERT	BooksCorpus + English Wikipedia	3.3B words
ModernBERT	Web, code, science	2T tokens
GPT-2	WebText (8M web pages)	40 GB

No human labels. The text itself gives every target.

Last layer: the same shape

BERT masked-LM head

h at [MASK] (768)

↓

Linear 768→768, GELU, LayerNorm

↓

Linear 768→30,522 (tied)

↓

softmax over 30,522 tokens

GPT-2 LM head

h at each position (768)

↓

LayerNorm

↓

Linear 768→50,257 (tied)

↓

softmax over 50,257 tokens

logit_v = h · e_v: one weight row per token = a **fixed menu**.

The last step as an equation

Head	Equation
BERT masked LM	logits = E h′ + b
BERT classifier	p = softmax(W C)
BERT multiple choice	s_k = v · C_k
GPT-2	logits_t = E · LN_f(h_t)

```
h’ = LayerNorm(GELU(W_t h + b_t)), C = final [CLS] vector
E = token embeddings (tied). Then: p = softmax(logits)
```

Toy example: BERT masked LM

`logits = E h′, p = softmax(logits)`

Input my dog is [MASK] → h′ = [0.2, 1.0, 0.8]

Token	Row of E	E h′	p
the	[1, 0, 0]	0.2	0.096
cute	[0, 1, 0]	1.0	0.214
hairy	[0, 1, 1]	1.8	0.476
apple	[1, 0, 1]	1.0	0.214

Shared: E · **Changes:** h′ per [MASK] · **Candidates:** the V rows

Toy example: GPT-2 generation

```
logits_t = E g_t, next = argmax softmax(logits_t)
```

```
E rows: Paris [1,0,0] · London [0,1,0] · . [0,0,1] · <end> [-1,-1,0]
```

Step	g_t	logits (P, L, ., end)	Pick
1	[1.2, 0.3, −0.5]	1.2, 0.3, −0.5, −1.5	Paris 0.604
2	[−0.4, −0.2, 1.4]	−0.4, −0.2, 1.4, 0.6	. 0.551
3	[−0.9, −0.8, 0.1]	−0.9, −0.8, 0.1, 1.7	<end> 0.736

Input grows: … France is → … is Paris → … Paris.

Shared: E · **Changes:** g_t each step · **Candidates:** the V rows

Inference: one pass vs a loop

BERT: all masks in one pass

`The capital of France is [MASK].` → **`Paris`**

GPT: one pass per new token

Pass	Input	New token
1	The capital of France is	Paris
2	… is Paris	.
3	… Paris.	<end>

Using each model to classify

BERT: add a head, fine-tune

`[CLS] Waste of money, battery died in 2 days. [SEP]`

h_CLS → **Linear(768 → 2)** → softmax: negative / positive

Classes are fixed at training: one weight row each.

GPT: prompt, then read logits

`Review: Waste of money, battery died in 2 days. Sentiment:`

Logits of **“positive”** and **“negative”** → softmax over 2

Any word works, but multi-token answers need more passes.

Toy example: BERT classifier

`p = softmax(W C), W: one row per class`

`[CLS] Waste of money, battery died in 2 days. [SEP]`

→ `C = [0.9, -0.6, 0.3]`

Class	Row of W	W C	p
negative	[1, −1, 0]	1.5	0.953
positive	[−1, 1, 0]	−1.5	0.047

`negative: 1×0.9 - 1×(-0.6) + 0×0.3 = 1.5`

Shared: W · **Changes:** C per input · **Candidates:** K fixed rows

BERT and GPT choose from a fixed menu of tokens.

A Jev-like model chooses from a menu you write in each request.

The task: a typed decision

Request

```
"state": "Help! I was charged twice for order #4471."
"question": "Which team handles this?"
"options": ["billing", "shipping", "technical"]
```

Response

```
"answer": "billing"
"probs": {"billing": 0.786, "shipping": 0.039, "technical": 0.175}
```

No text. The answer is **always one of your options**.

SWAG: a multiple-choice test

113k questions built from video captions. Pick the most likely next event from **4 endings**.

On stage, a woman takes a seat at the piano. She

a) sits on a bench as her sister plays with the doll.
b) smiles with someone as the music plays.
c) is in the crowd, watching the dancers.
d) nervously sets her fingers on the keys. (correct)

BERT-large **86.3%** · OpenAI GPT 78.0% · humans 88.0%

BERT on SWAG: one input per ending

```
input_k = [CLS] context [SEP] ending_k [SEP]
C_k = final [CLS] vector of input_k
s_k = v · C_k
p = softmax(s_1, s_2, s_3, s_4)
```

4 endings → 4 inputs → **4 passes**.

The only new weights: **one vector v**, shared by every ending. **K comes from the data.**

Toy example: BERT multiple choice

`s_k = v · C_k, v = [1, 0.5, -1]`

Pass	Ending	C_k	v · C_k	p
1	a) bench	[0.1, 0.2, 0.6]	−0.4	0.124
2	b) smiles	[0.3, 0.4, 0.5]	0.0	0.185
3	c) crowd	[0.2, −0.2, 0.4]	−0.3	0.137
4	d) keys	[0.9, 0.6, 0.1]	1.1	0.555

Shared: v · **Changes:** C_k, one pass each · **Candidates:** any K

The cost: context read K times

In BERT, the context also reads the option:

Pass 1: “piano” reads “**bench**”
Pass 4: “piano” reads “**keys**”

So **nothing can be shared.**

500-token context, 10 options × 5 tokens

BERT

5,050

10 × 505

Von

560

500 + 10 × 6

tokens processed

Von: all options in one sequence




```
Which team handles this? Help! I was charged twice for order #4471. [SEP]
[MASK] billing [MASK] shipping [MASK] technical
```

Backbone

ModernBERT-large, 395M

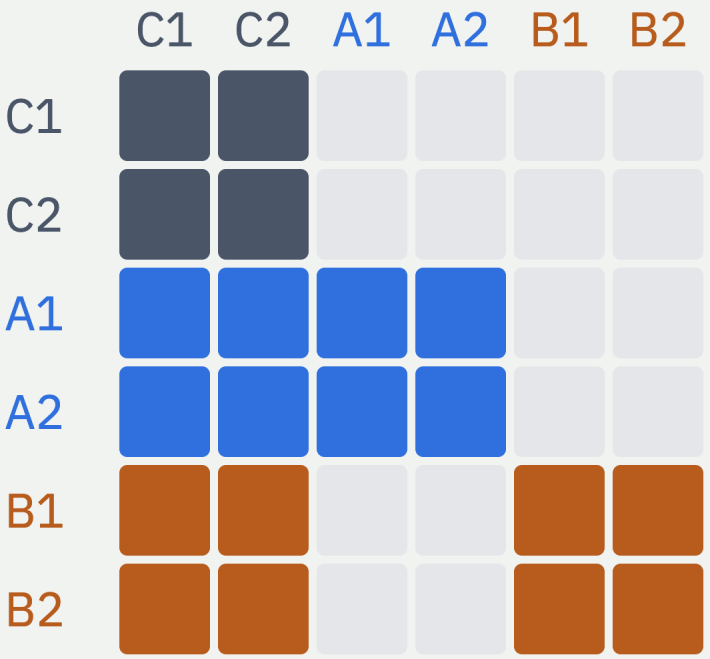
Readout

Vector at each [MASK]

Why [MASK]?

It learned to gather context

A mask keeps options apart



C = context

A = [MASK] billing · B = [MASK] shipping

1. Context reads **only context**.
2. An option reads **context + itself**.
3. Options **never read each other**.

Rule 1: the context is the same for every option. Von computes it **once**.

All options start at one position

Tokens	No reset	Von
team? twice [SEP]	0, 1, 2	0, 1, 2
[MASK] billing	3, 4	3, 4
[MASK] shipping	5, 6	3, 4
[MASK] technical	7, 8	3, 4

Why: RoPE encodes distance, so a later slot changes the score.

Evidence: shuffling flipped 49.5% of answers in v1.1, 0 in v1.2.

Von’s last layer: one scorer

`h_k (1024) → LN → Linear 1024→512 → GELU → LN → Linear 512→1 → z_k`

The **same** MLP runs on every [MASK]: K options → K logits → softmax.

$$u_k = \mathrm{LN}_2(\mathrm{GELU}(W_1\ \mathrm{LN}_1(h_k) + b_1))$$
$$z_k = w_2 \cdot u_k + b_2$$
$$p = \mathrm{softmax}(z / T)$$

W_1: 512 × 1024, w_2: 1 × 512

Toy example: Von

$z_k = w_2 \cdot u_k,\ w_2 = [1, -0.5, 0.5]$

All three u_k come from **the same pass**.

Option	u_k	$w_2 \cdot u_k$	p
billing	[1.5, −0.6, 0.4]	1.5 + 0.3 + 0.2 = 2.0	0.786
shipping	[−0.7, 0.6, 0.0]	−0.7 − 0.3 + 0 = −1.0	0.039
technical	[0.3, 0.2, 0.6]	0.3 − 0.1 + 0.3 = 0.5	0.175

Shared: scorer, context · **Changes:** h_k → u_k → z_k · **Candidates:** any K

How each head supports classes

Head	Weights	Passes	Classes
BERT masked LM	1 row per token	1	fixed (V)
GPT LM	1 row per token	1 per token	fixed (V)
BERT classifier	1 row per class	1	fixed (K)
BERT multiple choice	1 shared vector	K	any K
Von	1 shared scorer	1	any K

Any K = **a shared scorer** + **a vector from each option's text**.

Three question types, one head

Choice

billing · shipping · technical

p = 0.786, 0.039, 0.175

Answer: billing

Bool

“Yes, condition holds true.”
“No, condition is false.”

P(true) = p(Yes)

Score

0 low · 1 medium · 2 high

p = 0.1, 0.3, 0.6

Score = 1.5

Every type becomes a **choice**: one scorer, one softmax.

Training Von: one sample, one loss

```
"state": "Help! I was charged twice for order #4471."
"question": "Which team handles this?"
"options": ["billing", "shipping", "technical"]
"label": 0
```

Loss = CE + 0.5 × Brier

CE = -ln 0.786 = 0.241

Brier = $(0.786-1)^2 + 0.039^2 + 0.175^2 = 0.078$

Loss = 0.241 + 0.5 × 0.078 = 0.280

Data: ≈290k decision items

Calibration: a fitted temperature T

What Von gains and gives up

Gains

- Fewer tokens: 5,050 → 560
- Any K, no retraining
- Option order has no effect
- Answer is always an option

Limits

- Context cannot adapt per option
- Options compete only in softmax
- Still one pass per question
- Hard items: 0.373 vs Jev 0.741

Four steps from BERT to Von

1. BERT and GPT share blocks. The **mask** differs. Both end in a **fixed** menu of V rows.
2. A BERT classifier has one row per class, so its classes are fixed.
3. BERT multiple choice: one **shared** vector, any K, but K passes.
4. Von: [MASK] markers + mask + position reset → any K in **one pass**.

Read more

Blog posts

The Illustrated BERT, ELMo, and co. (Alammar)

Introducing ModernBERT (Hugging Face)

Papers

BERT (Devlin et al., 2018)

ModernBERT (Warner et al., 2024)

SWAG (Zellers et al., 2018)

GPT-2 (Radford et al., 2019)

Von

Source code on GitHub

Model card on Hugging Face

Note

TypeSafe has not published Jev's architecture. Von has the same interface, but it is not Jev.